Express.js — Day 14

Routes → Controllers → Services

Progress: 14 / 15 days completed

Goal: Learn how to organize an Express application into clear layers instead of keeping routing, HTTP handling, and business logic in one file.

1. Why Separate the Application?

Small Express examples can keep everything inside one route. As an application grows, that becomes difficult to read, test, debug, and maintain. Day 14 introduced separation of responsibilities.

```
Route
  ↓
Controller
  ↓
Service
  ↓
Controller
  ↓
HTTP Response
```

2. Route Layer

Responsibility: decide which HTTP method and URL should handle a request. The route should mainly forward the request to the appropriate controller.

```
router.post("/users", createUserController);
```

A route should not contain all the business logic for creating a user.

3. express.Router()

express.Router() creates a separate router object so related routes can live in their own file.

```
const express = require("express");
const createUserController =
    require("../controllers/userController");

const router = express.Router();

router.post("/users", createUserController);

module.exports = router;
```

4. Controller Layer

Responsibility: handle HTTP-level work. A controller receives req/res, gets the required input, calls the service, and sends the HTTP response.

```
const createUser = require("../services/userService");

const createUserController = (req, res) => {
    const user = createUser(req.body);
```

```
    res.status(201).json(user);
};

module.exports = createUserController;
```

The controller knows about req.body, HTTP status codes, and res.json(). It does not need to contain the actual user-creation business logic.

5. Service Layer

Responsibility: contain business logic. The service receives normal data, performs the operation, and returns a result. It normally should not depend on Express's req or res.

```
const createUser = (userData) => {
    const user = {
        id: 1,
        name: userData.name,
        age: userData.age
    };

    return user;
};

module.exports = createUser;
```

6. One Export vs Multiple Exports

Our service has one main function, so this is valid:

```
module.exports = createUser;
```

If a file exports several functions, an object is convenient:

```
module.exports = {
    createUser,
    deleteUser
};

const { createUser, deleteUser } =
    require("./userService");
```

Both styles are valid. The important difference is whether the exported value itself is the function or an object containing functions.

7. Day 14 Project Structure

```
day14/
■■■ controllers/
■   ■■■ userController.js
■■■ routes/
■   ■■■ userRoutes.js
■■■ services/
■   ■■■ userService.js
■■■ app.js
```

8. app.js — Connecting Everything

```
const express = require("express");
const userRoutes = require("./routes/userRoutes");
```

```
const app = express();

app.use(express.json());

app.use(userRoutes);

app.listen(3000, () => {
    console.log("Server is running on port 3000");
});
```

app.js mainly coordinates the application: Express setup, application middleware, route registration, and starting the server.

9. Why express.json() Is in app.js

Although the controller uses req.body, express.json() is registered in app.js because it is application-level middleware. It processes incoming JSON before the request reaches the route/controller.

```
Request
   ↓
app.js
   ↓
express.json()  ← parses JSON body
   ↓
Route
   ↓
Controller
   ↓
req.body is available
   ↓
Service
```

Because the middleware is registered before the routes, later request-handling layers can access the parsed req.body. The controller does not need to call express.json() itself.

10. The Debugging Lesson

Our first Postman request returned a 500 error: **Cannot read properties of undefined (reading 'name')**. The service was trying to read userData.name, but req.body was undefined.

```
createUser(req.body)
        ↓
req.body was undefined
        ↓
createUser(undefined)
        ↓
userData.name  ■
```

The cause was that app.js was missing:

```
app.use(express.json());
```

After adding it before app.use(userRoutes), the same Postman request successfully returned **201 Created** with the user object.

11. Complete Request Flow

```
POST /users
     ↓
app.js
     ↓
express.json()
     ↓
userRoutes
     ↓
createUserController
     ↓
createUser(req.body)
     ↓
userService
     ↓
return user
     ↓
controller
     ↓
res.status(201).json(user)
     ↓
Client
```

12. Layer Responsibilities

Layer	Main job
Routes	Match method + URL and forward the request.
Controllers	Handle HTTP request/response.
Services	Perform business logic and return results.
app.js	Configure the application, middleware, routes, and server.

13. Key Takeaways

• **Routes choose** where the request goes.

• **Controllers coordinate** the HTTP request and response.

• **Services perform** the business logic.

• **express.Router()** keeps related routes organized.

• **app.use(userRoutes)** registers the router with the application.

• **express.json()** parses incoming JSON so req.body is available.

• **201 Created** is appropriate when a new resource is successfully created.

14. Revision Questions

• What is the main responsibility of a route?

• What is the main responsibility of a controller?

• What belongs in a service?

• Why should a service normally not use req and res?

• What does express.Router() provide?

• Why is express.json() registered in app.js?

• Trace POST /users from the client to the final response.

• Why did our first Day 14 request fail with req.body undefined?

15. Rebuild This Years Later

Memory rule: Routes choose, Controllers handle HTTP, Services perform the work.

```
Route:
POST /users → controller

Controller:
req.body → service → res.status(201).json(...)

Service:
data → business logic → return result

App:
express.json() → routes → listen
```

If you can reconstruct this flow and explain why each layer exists, you understand the architecture instead of merely memorizing the code.

DAY 14 COMPLETE ■